

11

Monads

In the previous chapter, we introduced a simple algebraic structure, the monoid. This was our first instance of a completely abstract, purely algebraic interface, and it led us to think about interfaces in a new way. A useful interface may be defined only by a collection of operations related by laws.

In this chapter, we'll continue this mode of thinking and apply it to the problem of factoring out code duplication across some of the libraries we wrote in parts 1 and 2. We'll discover two new abstract interfaces, `Functor` and `Monad`, and get more general experience with spotting these sorts of abstract structures in our code.¹

11.1 *Functors: generalizing the map function*

In parts 1 and 2, we implemented several different combinator libraries. In each case, we proceeded by writing a small set of primitives and then a number of combinators defined purely in terms of those primitives. We noted some similarities between derived combinators across the libraries we wrote. For instance, we implemented a `map` function for each data type, to lift a function taking one argument “into the context of” some data type. For `Gen`, `Parser`, and `Option`, the type signatures were as follows:

```
def map[A,B] (ga: Gen[A]) (f: A => B): Gen[B]

def map[A,B] (pa: Parser[A]) (f: A => B): Parser[B]

def map[A,B] (oa: Option[A]) (f: A => B): Option[B]
```

¹ The names *functor* and *monad* come from the branch of mathematics called *category theory*, but it isn't necessary to have any category theory background to follow the content in this chapter. You may be interested in following some of the references in the chapter notes for more information.

These type signatures differ only in the concrete data type (Gen, Parser, or Option). We can capture as a Scala trait the idea of “a data type that implements map”:

```
trait Functor[F[_]] {
  def map[A,B](fa: F[A])(f: A => B): F[B]
}
```

Here we’ve parameterized map on the type constructor, `F[_]`, much like we did with `Foldable` in the previous chapter.² Instead of picking a particular `F[_]`, like `Gen` or `Parser`, the `Functor` trait is parametric in the choice of `F`. Here’s an instance for `List`:

```
val listFunctor = new Functor[List] {
  def map[A,B](as: List[A])(f: A => B): List[B] = as map f
}
```

We say that a type constructor like `List` (or `Option`, or `F`) is a functor, and the `Functor[F]` instance constitutes proof that `F` is in fact a functor.

What can we do with this abstraction? As we did in several places throughout this book, we can discover useful functions just by *playing* with the operations of the interface, in a purely algebraic way. You may want to pause here to see what (if any) useful operations you can define only in terms of `map`.

Let’s look at one example. If we have `F[(A, B)]` where `F` is a functor, we can “distribute” the `F` over the pair to get `(F[A], F[B])`:

```
trait Functor[F[_]] {
  ...
  def distribute[A,B](fab: F[(A, B)]): (F[A], F[B]) =
    (map(fab)(_._1), map(fab)(_._2))
}
```

We wrote this just by following the types, but let’s think about what it *means* for concrete data types like `List`, `Gen`, `Option`, and so on. For example, if we distribute a `List[(A, B)]`, we get two lists of the same length, one with all the `As` and the other with all the `Bs`. That operation is sometimes called *unzip*. So we just wrote a generic `unzip` function that works not just for lists, but for any functor!

And when we have an operation on a product like this, we should see if we can construct the opposite operation over a sum or coproduct:

```
def codistribute[A,B](e: Either[F[A], F[B]]): F[Either[A, B]] =
  e match {
    case Left(fa) => map(fa)(Left(_))
    case Right(fb) => map(fb)(Right(_))
  }
```

What does `codistribute` mean for `Gen`? If we have either a generator for `A` or a generator for `B`, we can construct a generator that produces either `A` or `B` depending on which generator we actually have.

² Recall that a *type constructor* is applied to a type to produce a type. For example, `List` is a type constructor, not a type. There are no values of type `List`, but we can apply it to the type `Int` to produce the type `List[Int]`. Likewise, `Parser` can be applied to `String` to yield `Parser[String]`.